

Supplementary material - Read me

Description

The present document aims to provide a more detailed description of the procedures followed for the implementation of our custom CFD adsorption model in ANSYS Fluent by means of User-Defined Functions (UDFs).

Requirements

- Our model was developed in ANSYS Fluent v19.2. Subsequent versions of the software should be compatible with the steps detailed next.
- A C compiler is required to integrate the UDFs into ANSYS Fluent. A GNU GCC compiler is recommended.

Procedure

1. Implementation of adsorption transport terms into Fluent's transport equations

- The additional adsorption terms in the transport equations must be implemented as source terms. The mathematical expressions for these are provided in the paper.
- All the parameters and equations that define a given source term are coded within a DEFINE_SOURCE macro.
- At least one DEFINE_SOURCE macro should be used for each transport equation: species, energy, and continuity equation. Thus, they should be named appropriately in order to easily identify them later during the hooking procedure in Fluent.
- All the necessary variables must be declared. One of them -the **source** variable- will hold the numerical value of the source term after all the calculations are performed.
- At the end, the value of the **source** variable is returned. This is the value that Fluent will add to the corresponding transport equation at every iteration.
- Attention must be paid to the signs of the adsorption terms. If a term is negative (which would technically make it a sink term), then a minus sign must precede the mathematical definition of the **source** variable.
- All the source terms are added to the right-hand side of the transport equations. Hence, the sign of the **source** variable should be consistent with this convention.

```

/***** SOURCE UDF MACRO FOR THE CONTINUITY EQUATION *****/
DEFINE_SOURCE(uhf_source_term_masscontinuity,c,t,dS,eqn)
{
    /* Definition and declaration of variables */
    int i;
    double dq1dt, dq2dt;
    double source;

    /* We call the relevant variables for this source term, from Fluent */
    dq1dt = C_UDMI(c,t,2);
    dq2dt = C_UDMI(c,t,3);

    /* Formulation of equations */

    source = -((1-poro_fb)/1.00)*rho_p*(Mm1_kg*dq1dt + Mm2_kg*dq2dt);

    /* We determine the derivative dS/dphi, in order to aid the convergence (optional) */
    dS[eqn] = 0.0;

    /* The adsorption continuity source term (or rather sink term) is returned */
    return source;
}

```

Figure 1. Code snippet for the adsorption source term in the continuity equation. A similar structure is used for the rest of the transport equations.

2. Implementation of a custom LDF numerical solver

- The LDF model constitutes a separate transport equation. It is an ordinary differential equation that cannot be solved by separation of variables since its parameters and coefficients are not constant. A numerical solver is thus required.
- An implicit trapezoidal rule (2nd order accuracy) was used for implementing the solver. Any other numerical method may be used.
- The numerical solver is coded into a DEFINE_ADJUST macro, which is executed first and before the remaining transport equations.
- If higher numerical stability is required, then relaxation factors for the LDF equation can be implemented within the same DEFINE_ADJUST macro.

```

/* ----- Actual IMPLICIT Method START ----- */

/* q1_new = (q1_M1 + h*k1*q1star)/(1 + h*k1); */
q1_new = (2*q1_M1 + h*k1*q1star + h*f1_M1)/(2 + h*k1);
q1_new = RLX*q1_new + (1 - RLX)*C_UDMI(c,t,0);

/* q2_new = (q2_M1 + h*k2*q2star)/(1 + h*k2); */
q2_new = (2*q2_M1 + h*k2*q2star + h*f2_M1)/(2 + h*k2);
q2_new = RLX*q2_new + (1 - RLX)*C_UDMI(c,t,1);

/* NOTE: qi_new can be interpreted as qi_current */

/* ----- Actual RK Method END ----- */

```

Figure 2. Trapezoidal method and under-relaxation factors.

- The main output from this code are the solid phase concentrations q_i and the adsorption rate $\partial q_i / \partial t$. The latter is used in nearly all the adsorption transport source terms.
- These variables are stored in a User-Defined Memory (UDMI), as shown:

```
/* Update memory with the new value for the variables */
C_UDMI(c,t,0) = q1_new;
C_UDMI(c,t,1) = q2_new;
C_UDMI(c,t,2) = dq1dt;
C_UDMI(c,t,3) = dq2dt;
```

Figure 3. Definition of User-Defined Memories.

3. Compilation of UDFs in ANSYS Fluent

- At this point, the UDF scripts have all adsorption-related mathematical information.
- This information must be 'translated' from C programming language and passed to ANSYS. To do this, a C compiler is needed.
- In Windows, in order to launch ANSYS with enabled compatibility for C compilation, Visual Studio is required. When installed, Fluent needs to be launched using the x86_x64 Cross Tools Command Prompt.
- Once there, the following three commands are required:

```
C:\> x86_x64 Cross Tools Command Prompt for VS 2019 - fluent
*****
** Visual Studio 2019 Developer Command Prompt v16.11.2
** Copyright (c) 2021 Microsoft Corporation
*****
[vcvarsall.bat] Environment initialized for: 'x86_x64'

C:\Program Files (x86)\Microsoft Visual Studio\2019\Community>cd\

C:\>cd C:\Program Files\ANSYS Inc\v192\fluent\ntbin\win64

C:\Program Files\ANSYS Inc\v192\fluent\ntbin\win64>fluent
```

Figure 4. Initializing Fluent with C compiler compatibility.

- Linux users shouldn't require to do the last two steps.
- Next, to compile the scripts and integrate them into ANSYS Fluent, the user should go to the *User Defined* menu > *Functions* > *Compiled*.
- There, we select the necessary UDF scripts, click *Build*, and *Load*.

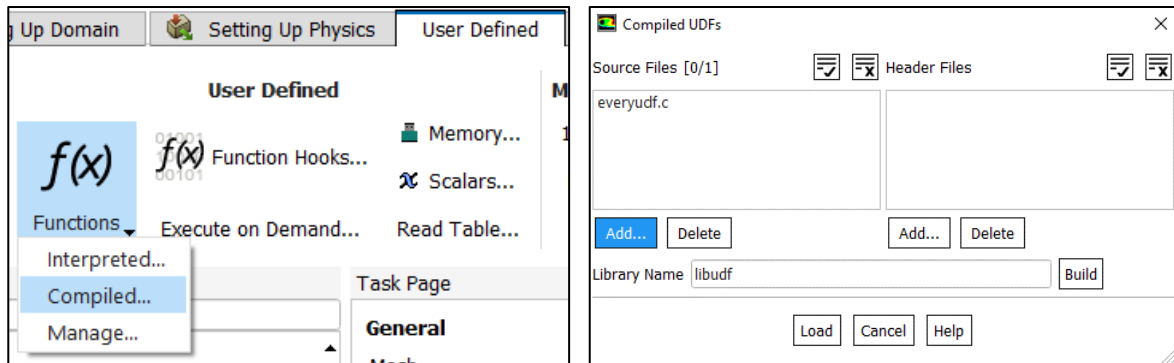


Figure 5. Compiling UDFs.

4. Hooking adsorption UDFs into ANSYS Fluent

- Upon successful compilation and loading, each of the macros must be linked to their corresponding components next:
- To hook the source terms, we go into *Cell Zone Conditions* and enable *Source Terms* (see Figure 6).
- Then, we switch to the *Source Terms* tab, select the corresponding transport equation (Mass source, Energy Source, etc.), and point to the respective UDF macro (see Figure 7).

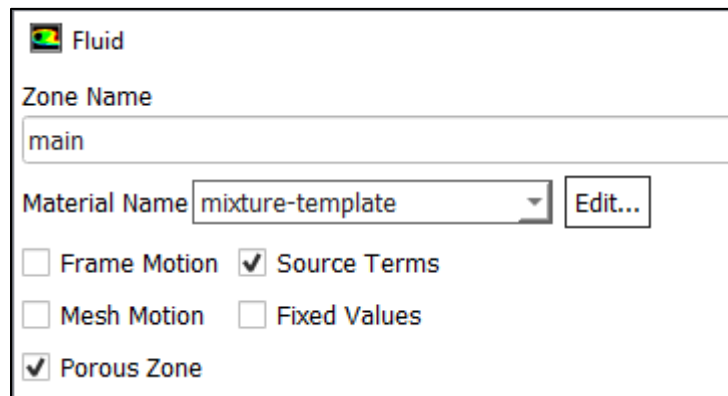


Figure 6. Enabling source terms.

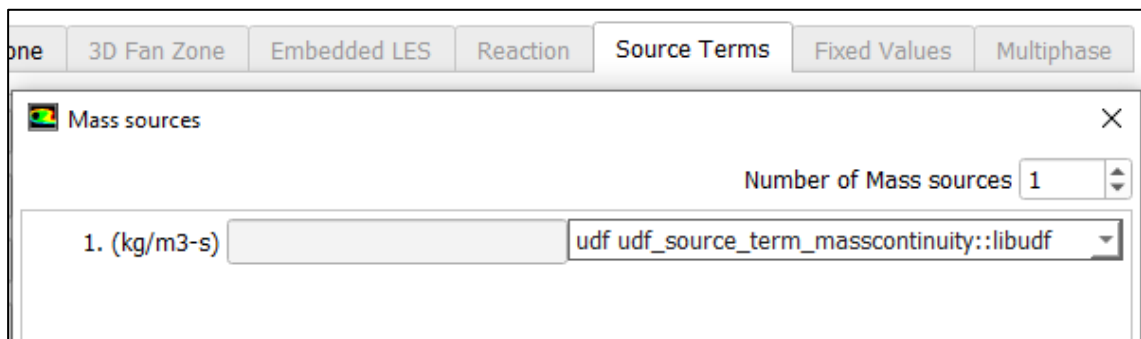


Figure 7. Pointing to the corresponding UDF source macro.

- To hook the LDF numerical solver, we go to the *User Defined* menu > *Function hooks*, where the `DEFINE_ADJUST` and all its accessory macros can be hooked:

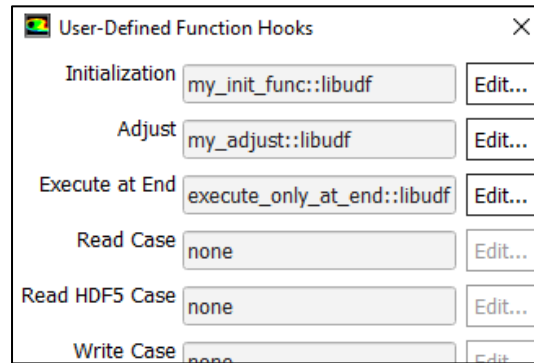


Figure 8. Hooking the necessary macros for the LDF solver.

5. Setting up the case file

- The rest of the procedure follows the same workflow as setting up a standard Fluent simulation.
- *Transient* and *Axisymmetric* are selected from the *General* task page.
- A pressure-based solver is used.
- In *Models*, the *Energy*, *Viscous Model (Laminar)*, and *Species Model* are enabled.
- The necessary materials are specified in the *Materials* tree.
- In *Cell Zone Conditions*, we enable *Porous Zone* to model the flow across the packed bed. The inverse-permeability values for the packed bed must be specified, along with the properties of the porous material (refer to Figure 9).

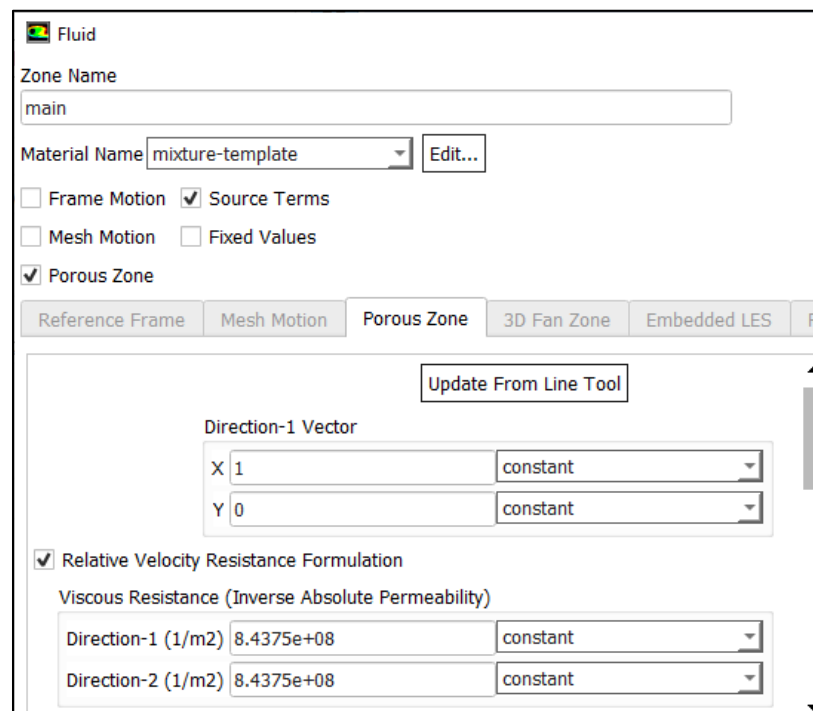
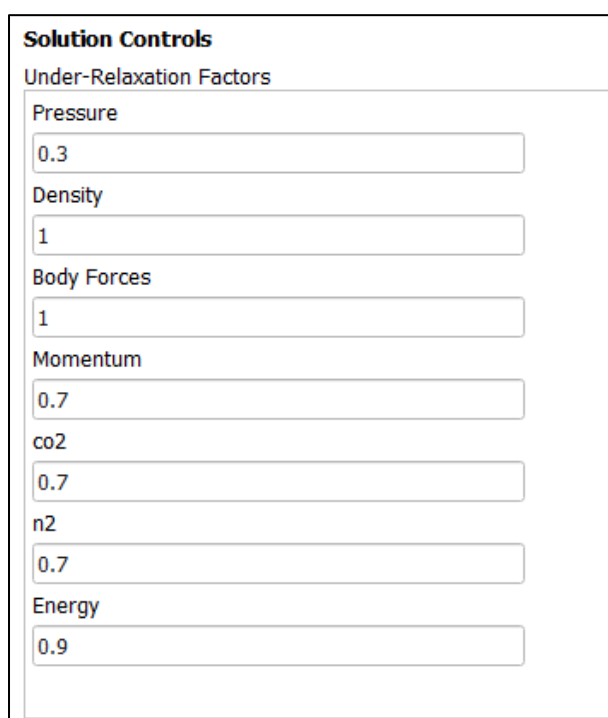


Figure 9. Cell zone settings.

- In the *Boundary Conditions* tree, the inlet composition and velocity are specified. An outflow outlet boundary condition was used.
- The top boundaries are set to *wall* and the bottom boundaries are set to *axis*.
- No-slip condition and zero material flux are set in the wall boundary conditions. The user should also setup the thermal boundary condition according to their heat transfer requirements.
- In *Methods*, higher order discretization schemes are recommended for improved accuracy.
- The under-relaxation factors are setup in *Solution Controls*. Appropriate values here aid with stabilization of the solution. For the Dynamic Column Breakthrough simulations, the following values have been found to be suitable:



Solution Controls

Under-Relaxation Factors

Pressure	0.3
Density	1
Body Forces	1
Momentum	0.7
co2	0.7
n2	0.7
Energy	0.9

Figure 10. Recommended under-relaxation factors

- The user can choose to enable *Report Definitions* to set up a monitor that will print out the plot of any variable as the simulation progresses.
- The residual criterion is configured in *Report Definitions > Residuals*.
- The initial values for each of the main transport variables is specified in *Initialization*. The *Standard Initialization* method is selected, and the following values are used for the N₂/He Dynamic Column Breakthrough cases:

Initial Values

Gauge Pressure (pascal)
0

Axial Velocity (m/s)
1e-08

Radial Velocity (m/s)
0

Temperature (k)
296.15

co2
1.802816e-08

n2
1e-08

Figure 11. Initialization of the problem.

- Note that the initial values of solid phase loading q_i are specified within the UDF in the DEFINE_INIT macro (refer to the code for further details).
- In the *Calculation Activities* tree, we can setup Autosave. This will generate .cas & .dat files every given number of time steps, which is useful for post-processing purposes.

Autosave

Save Data File Every (Time Steps) 10

Data File Quantities...

Save Associated Case Files

☐ Only if Modified

☒ Each Time

File Storage Options

☐ Retain Only the Most Recent Files

Maximum Number of Data Files 0

Only Associated Case Files are Retained

File Name

I:\10\Project\Fluent Files\Adsorption_Column.cas Browse...

Append File Name with flow-time

Decimal Places in File Name 6

OK Cancel Help

Figure 12. Setting Autosave.

- Finally, in *Run Calculation*, we can setup the time step size, number of time steps and maximum number of iterations per time step.

6. Post-processing

- For the procedures described above, the post-processing workflow may be carried out in two ways:
- Monitored variables: If the user enables *Report Definitions*, Fluent will create *.out* files with information on variables at every time step. These can then be processed in a spreadsheet or software like MATLAB.
- Data files: The user may use the *.cas* & *.dat* files created in *Autosave* and import them into any CFD post-processor, such as Tecplot 360 or CFD-Post. This allows further extraction of data, and the creation of 2D/3D plots (such as contour profiles) and animations.